

Claude Code Desktop 配置第三方 API 成功经验 Runbook

本机 HTTPS 代理、自签名证书、模型槽位映射、Cowork 证书修复与 macOS 可迁移排障手册

整理日期：2026-05-14。本文刻意不包含任何真实 API key、token、密码或私钥；分享或迁移时请使用占位符。

1. 一句话结论

Claude Code Desktop 新版本配置第三方 API 时，CLI 可用不代表 App 全部可用。稳定方案是让 Desktop 3P、Code host、CLI 都统一指向本机 HTTPS 代理；macOS 信任本地 CA；代理负责模型槽位映射；Cowork/host loop 通过 launcher 强制获得证书环境。

- Desktop 3P 主进程使用 Electron 网络栈，需要系统或登录钥匙串信任本地 CA。
- Code 模式会调用 Claude Code host binary，并可能读取 `~/.claude/settings.json`。
- Cowork 可能不完整继承 CLI 环境变量，证书失败时需要 host binary launcher 兜底。
- `v1/models` discovery 返回 0 usable models 不一定阻断，只要 `inferenceModels` 已显式配置且 API 请求 200。
- `server is busy` 有时只是 UI 层泛化错误，真实原因要看 `main.log` 和代理日志。

2. 最终成功架构

最终链路分为 Desktop 3P UI、Desktop Code host/Cowork、CLI 三条入口，但全部收敛到同一个本机 HTTPS 代理。

```
Claude Code Desktop 3P UI
-> https://127.0.0.1:38443
-> local HTTPS proxy
-> https://ark.cn-beijing.volces.com/api/coding

Claude Code host binary / Cowork
-> Claude-3p/claude-code/<version>/claude-ca-launcher
-> NODE_USE_SYSTEM_CA=1
-> NODE_EXTRA_CA_CERTS=/path/to/ca.crt
-> https://127.0.0.1:38443
-> local HTTPS proxy
-> Ark Anthropic-compatible endpoint

Claude Code CLI
-> ~/.claude/settings.json
-> ANTHROPIC_BASE_URL=https://127.0.0.1:38443
-> local HTTPS proxy
-> Ark Anthropic-compatible endpoint
```

Claude 槽位	App/CLI 发送模型名	代理实际转发模型	用途
Opus	claude-opus-4-6	glm-5.1	大模型槽位
Sonnet	claude-sonnet-4-6	kimi-k2.6	中模型/默认槽位
Haiku	claude-haiku-4-5	doubao-seed-2.0-pro	小模型槽位

3. 本次成功路径

1. 准备本机 HTTPS 代理，监听 `https://127.0.0.1:38443`，上游为 `https://ark.cn-beijing.volces.com/api/coding`。
2. 生成本地 CA 和 server certificate，server certificate 的 SAN 必须包含 `127.0.0.1`。
3. 把本地 CA 加入 macOS Keychain 并设置 SSL trust，确认普通 `curl https://127.0.0.1:38443/health` 可以成功。
4. 用 LaunchAgent 托管代理，确保登录后自动启动。

5. Claude Desktop Developer 模式中配置 third-party inference：Gateway base URL 指向本机代理，Model list 写 Claude 槽位名。
6. 更新 `~/claude/settings.json`：只保留环境变量配置，删除 `ANTHROPIC\_MODEL` 和 `modelOverrides`。
7. 根据 Desktop 日志里的 `[CCD] Initialized with version ...` 创建 host binary 版本目录，写入 `verified`。
8. 如果 Desktop host binary 下载失败，把对应路径软链到本机 CLI；如果 Cowork 证书失败，则改为软链到 `claude-ca-launcher`。
9. 用 Desktop 日志、代理日志、CLI 极简环境测试共同确认成功。

4. 关键环境与路径

项目	建议值或示例	说明
本机代理	https://127.0.0.1:38443	Desktop 3P、host、CLI 统一入口。
上游接口	https://ark.cn-beijing.volces.com/api/coding	Anthropic-compatible gateway。
代理目录	/path/to/claude-local-proxy	不同电脑按实际项目路径替换。
LaunchAgent	~/Library/LaunchAgents/com.cj.claude-local-https-proxy.plist	负责持续运行代理。
Desktop 3P 配置库	~/Library/Application Support/Claude-3p/configLibrary	_meta.json 的 appliedId 必须指向有效配置 JSON。
Desktop mode	~/Library/Application Support/Claude-3p/claude_desktop_config.json	至少应启用 deploymentMode: 3p。
CLI settings	~/claude/settings.json	Code host/CLI 的环境变量入口。
host binary	~/Library/Application Support/Claude-3p/claude-code/<version>/..	版本号以 Desktop 日志为准，不要猜。
证书 launcher	/path/to/claude-local-proxy/bin/claude-ca-launcher	Cowork 证书失败时使用。

5. 本机代理配置

代理负责三件事：提供 Desktop 能访问的 HTTPS endpoint；把请求转发到 Ark；把 Claude 槽位模型名改写为真实上游模型。

```
const bigModel = process.env.BIG_MODEL || 'glm-5.1';
const middleModel = process.env.MIDDLE_MODEL || 'kimi-k2.6';
const smallModel = process.env.SMALL_MODEL || 'doubao-seed-2.0-pro';

function mappedModel(model) {
  if (typeof model !== 'string') return model;
  const modelWithoutContextSuffix = model.replace(/\[[^\]]+\]$/, '');
  const normalized = modelWithoutContextSuffix.toLowerCase();
  if (normalized.includes('haiku')) return smallModel;
  if (normalized.includes('opus')) return bigModel;
  if (normalized.includes('sonnet') || normalized.includes('claude')) return middleModel;
  return modelWithoutContextSuffix;
}

curl --silent --show-error \
  --cacert /path/to/claude-local-proxy/certs/ca.crt \
  https://127.0.0.1:38443/health

# 期望输出
{
  "ok": true,
  "upstream": "https://ark.cn-beijing.volces.com/api/coding",
  "bigModel": "glm-5.1",
  "middleModel": "kimi-k2.6",
  "smallModel": "doubao-seed-2.0-pro"
}
```

6. 自签名证书与 macOS 信任

证书是这次最关键的差异点：CLI 可以通过 `NODE\_EXTRA\_CA\_CERTS` 或 curl `--cacert` 信任本地 CA，但 Desktop/Electron 和 Cowork/host loop 还需要系统或登录钥匙串能建立可信链。

```
mkdir -p certs

openssl genrsa -out certs/ca.key 4096
openssl req -x509 -new -nodes \
  -key certs/ca.key \
  -sha256 -days 825 \
  -out certs/ca.crt \
  -subj "/CN=CJ Claude Local Proxy CA"

cat > openssl-server.cnf <<'EOF'
[req]
default_bits = 2048
prompt = no
default_md = sha256
distinguished_name = dn
req_extensions = req_ext

[dn]
CN = localhost

[req_ext]
subjectAltName = @alt_names

[alt_names]
DNS.1 = localhost
IP.1 = 127.0.0.1
IP.2 = ::1
EOF

openssl genrsa -out certs/server.key 2048
openssl req -new \
  -key certs/server.key \
  -out certs/server.csr \
  -config openssl-server.cnf

openssl x509 -req \
  -in certs/server.csr \
  -CA certs/ca.crt \
  -CAkey certs/ca.key \
  -CAcreateserial \
  -out certs/server.crt \
  -days 825 -sha256 \
  -extensions req_ext \
  -extfile openssl-server.cnf

# 当前用户 login keychain
security add-trusted-cert \
  -r trustRoot \
  -p ssl \
  -k ~/Library/Keychains/login.keychain-db \
  /path/to/claude-local-proxy/certs/ca.crt

# 如果需要 System keychain, 必须在目标 Mac 本机交互式 sudo; 纯 SSH 可能被 macOS 授权拦截
sudo security add-trusted-cert \
  -d -r trustRoot \
  -p ssl \
  -k /Library/Keychains/System.keychain \
  /path/to/claude-local-proxy/certs/ca.crt

security verify-cert \
  -c /path/to/claude-local-proxy/certs/server.crt \
  -p ssl \
  -s 127.0.0.1
```

如果通过 SSH 执行 System keychain 信任时报 `The authorization was denied since no user interaction was possible.`，这不是证书命令写错，而是 macOS 不允许无交互授权。需要用户在目标 Mac 本机执行，或用 MDM/配置描述文件下发。

7. LaunchAgent 自动启动代理

LaunchAgent 要写绝对路径，尤其是 Node 路径、工作目录、证书路径。Apple Silicon 常见 Node 路径可能是 `/opt/homebrew/bin/node` 或 `/usr/local/bin/node`，用 `command -v node` 确认。

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
  "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>Label</key>
  <string>com.cj.claude-local-https-proxy</string>
  <key>ProgramArguments</key>
  <array>
    <string>/usr/local/bin/node</string>
    <string>/path/to/claude-local-proxy/server.js</string>
  </array>
  <key>WorkingDirectory</key>
  <string>/path/to/claude-local-proxy</string>
  <key>RunAtLoad</key>
  <true/>
  <key>KeepAlive</key>
  <true/>
  <key>EnvironmentVariables</key>
  <dict>
    <key>LISTEN_HOST</key><string>127.0.0.1</string>
    <key>LISTEN_PORT</key><string>38443</string>
    <key>UPSTREAM_BASE_URL</key><string>https://ark.cn-beijing.volces.com/api/coding</string>
    <key>BIG_MODEL</key><string>glm-5.1</string>
    <key>MIDDLE_MODEL</key><string>kimi-k2.6</string>
    <key>SMALL_MODEL</key><string>doubao-seed-2.0-pro</string>
    <key>TLS_CERT_FILE</key><string>/path/to/claude-local-proxy/certs/server.crt</string>
    <key>TLS_KEY_FILE</key><string>/path/to/claude-local-proxy/certs/server.key</string>
  </dict>
  <key>StandardOutPath</key>
  <string>/path/to/claude-local-proxy/logs/proxy.log</string>
  <key>StandardErrorPath</key>
  <string>/path/to/claude-local-proxy/logs/proxy.err.log</string>
</dict>
</plist>

launchctl unload ~/Library/LaunchAgents/com.cj.claude-local-https-proxy.plist 2>/dev/null || true
launchctl load ~/Library/LaunchAgents/com.cj.claude-local-https-proxy.plist
launchctl kickstart -k gui/${id -u} /com.cj.claude-local-https-proxy
launchctl print gui/${id -u} /com.cj.claude-local-https-proxy
```

8. Desktop 3P 配置

Developer 模式里的 Configure third-party inference 最终会落到 `~/Library/Application

Support/Claude-3p/configLibrary`。配置可以用 UI 写，也可以脚本生成，但结构要保持一致。

```
{
  "provider": "gateway",
  "gatewayBaseUrl": "https://127.0.0.1:38443",
  "gatewayApiKey": "<ARK_API_KEY>",
  "gatewayAuthScheme": "bearer",
  "inferenceModels": [
    { "id": "claude-sonnet-4-6", "name": "Sonnet 4.6" },
    { "id": "claude-opus-4-6", "name": "Opus 4.6" },
    { "id": "claude-haiku-4-5", "name": "Haiku 4.5" }
  ],
  "hideAnthropicSignIn": true
}
```

注意：如果 `inferenceModels` 显式配置，日志里 `Gateway /v1/models returned 0 usable models` 不一定是失败；真正要看 health 是否 healthy，以及 `/v1/messages` 是否返回 200。

9. CLI settings 只保留环境变量

`model: "haiku"` 这类字段是 CLI 选择模型后的回写状态，可能被 App 共享读取。真正影响请求目标的是环境变量。最稳妥做法是删除强覆盖模型的 `ANTHROPIC\_MODEL`，只保留默认槽位和代理地址。

```
{
  "env": {
    "ANTHROPIC_BASE_URL": "https://127.0.0.1:38443",
    "ANTHROPIC_AUTH_TOKEN": "<ARK_API_KEY>",
    "ANTHROPIC_DEFAULT_OPUS_MODEL": "claude-opus-4-6",
    "ANTHROPIC_DEFAULT_SONNET_MODEL": "claude-sonnet-4-6",
    "ANTHROPIC_DEFAULT_HAIKU_MODEL": "claude-haiku-4-5",
    "NODE_USE_SYSTEM_CA": "1",
    "NODE_EXTRA_CA_CERTS": "/path/to/claude-local-proxy/certs/ca.crt",
    "SSL_CERT_FILE": "/path/to/claude-local-proxy/certs/ca.crt"
  }
}
```

- 不要保留 `ANTHROPIC\_MODEL=glm-5.1`，它会覆盖 App/CLI 的槽位选择。
- 不要再写 `modelOverrides`，否则会和管理里的槽位映射形成两层映射，排障更难。
- 默认槽位要写 Claude 名称，不要写真实上游模型名。
- API key 用 `<ARK\_API\_KEY>` 占位；真实值只进入本机配置，不进入文档或仓库。

10. Desktop host binary 与 `.verified`

Claude Desktop 会维护自己的 Claude Code host binary。版本号不是 CLI 当前版本，而是 Desktop 日志中 `[CCD] Initialized with version ...` 的值。路径错一个版本，App 仍会下载或 repair。

```
# 先关闭 Claude 和 Claude Helper
osascript -e 'quit app "Claude"' || true
pkill -f "Claude Helper" || true

# 从日志确定版本
grep -E "\[CCD\] Initialized with version" "$HOME/Library/Logs/Claude-3p/main.log" | tail

VERSION=2.1.138
BASE="$HOME/Library/Application Support/Claude-3p/claude-code/$VERSION"
mkdir -p "$BASE/claude.app/Contents/MacOS"
touch "$BASE/.verified"

# 简单兜底：指向本机 CLI
ln -sfn "$(command -v claude)" "$BASE/claude.app/Contents/MacOS/claude"
ln -sfn "$(command -v claude)" "$BASE/claude"
```

如果只建软链但没有 `.verified`，Desktop repair/download 流程可能在下载超时后清空目录。远端 Mac 上曾观察到 `downloads.claude.ai` 的 darwin/linux 资源下载超时，因此 `.verified` 是必要保护。

11. Cowork 证书失败的最终修复

Cowork 的 UI 可能显示 `server is busy`，但 transcript 或日志里的真实错误是 `API Error: Unable to connect to API: SSL certificate verification failed`。这说明模型 API 上游未必有问题，而是 host loop 没拿到 CA 环境。

```
cat > /path/to/claude-local-proxy/bin/claude-ca-launcher.c <<'EOF'
#include <unistd.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    setenv("NODE_USE_SYSTEM_CA", "1", 1);
    setenv("NODE_EXTRA_CA_CERTS", "/path/to/claude-local-proxy/certs/ca.crt", 1);
    setenv("SSL_CERT_FILE", "/path/to/claude-local-proxy/certs/ca.crt", 1);

    char *target = "/opt/homebrew/bin/claude";
    execv(target, argv);
    return 127;
}
EOF
```

```
clang -O2 -arch arm64 \
  /path/to/claude-local-proxy/bin/claude-ca-launcher.c \
  -o /path/to/claude-local-proxy/bin/claude-ca-launcher
chmod +x /path/to/claude-local-proxy/bin/claude-ca-launcher

VERSION=2.1.138
BASE="$HOME/Library/Application Support/Claude-3p/claude-code/$VERSION"
ln -sfn /path/to/claude-local-proxy/bin/claude-ca-launcher "$BASE/claude.app/Contents/MacOS/claude"
ln -sfn /path/to/claude-local-proxy/bin/claude-ca-launcher "$BASE/claude"

# 极简环境测试：不继承 shell 里的证书变量，仍应能返回 ok
env -i HOME="$HOME" PATH="/usr/bin:/bin:/usr/sbin:/sbin:/opt/homebrew/bin:/usr/local/bin" \
  "$HOME/Library/Application Support/Claude-3p/claude-code/2.1.138/claude.app/Contents/MacOS/claude" \
  -p '只回复 ok'
```

12. 成功信号

检查点	命令/位置	成功信号
端口监听	ls -l /path/to/claude-local-proxy/bin/claude-ca-launcher	存在 node 或代理进程监听。
代理健康	curl https://127.0.0.1:38443/health	系统信任后不带 --cacert 也成功。
Desktop health	~/Library/Logs/Claude-3p/main.log	ConfigHealth recomputed { state: 'healthy', provider: 'gateway' }
模型列表	代理日志	GET /v1/models?limit=1000 -> 200
聊天请求	代理日志	POST /v1/messages -> 200 或 POST /v1/messages?beta=true -> 200
token 计数	代理日志	POST /v1/messages/count_tokens?beta=true -> 200
模型映射	代理日志	mapped model claude-sonnet-4-6 -> kimi-k2.6 等。
Cowork 证书	极简环境调用 launcher	能返回 ok，不再报 SSL certificate verification failed。

13. 失败路径总结

失败路径	现象	根因	处理方式
Desktop 直接指向 Ark	CLI 可用，App 503 或证书错误	Desktop 3P 网络栈和 CLI 不同	改为本机 HTTPS 代理。
本机代理使用 HTTP	Desktop 新版本不接受或不稳定	新版本要求本地代理为 HTTPS	使用自签 server certificate。
只给 curl/CLI 配 CA	CLI 正常，Desktop 报 cert authority invalid	Electron 不继承 CLI CA 文件	把 CA 加入 Keychain 并设置 SSL trust。
保留 ANTHROPIC_MODEL	App 选择 Sonnet/Haiku 仍显示或调用 glm	强制模型覆盖了槽位选择	删除 ANTHROPIC_MODEL。
同时保留 modelOverrides	模型映射难以判断	CLI 映射和代理映射叠加	只保留代理映射。
host binary 版本目录写错	App 仍下载或报 Code host 不可用	Desktop 使用内置版本而非本机 CLI 版本	按 [CCD] Initialized with version ... 创建目录。
没有 .verified	手工目录被清空	repair/download 流程认为目录未完成	创建 .verified 后再放 binary。
Cowork 报 server is busy	Code 可对话，Cowork 失败	host loop 实际证书失败	安装 claude-ca-launcher。
远端 SSH 信任 System keychain	authorization denied	macOS 需要交互授权	目标 Mac 本机执行 sudo 或用 MDM。
downloads.claude.ai 超时	VM boot failed / bash proxy unavailable	网络下载资源失败	配置网络代理、白名单或离线缓存。

14. macOS 迁移清单

10. 确认架构与路径：`uname -m`、`command -v node`、`command -v claude`、Claude Desktop 版本。
11. 复制或创建项目目录，但不要复制旧机器的 `server.key`；推荐在目标机器重新生成证书。
12. 写 LaunchAgent 并启动代理。
13. 在目标机器本机信任 CA，必要时让用户交互式输入 sudo。
14. 写 Desktop 3P config 和 `\_meta.json`，Gateway 指向 `https://127.0.0.1:38443`。
15. 写 `~/claude/settings.json`，只保留环境变量。
16. 启动 Claude Desktop，读取 `[CCD] Initialized with version ...`，创建对应 host binary 目录和 `.verified`。
17. 如果 Cowork 仍报证书问题，编译并安装 `claude-ca-launcher`。
18. 按成功信号逐项验证。

15. 分享和入库注意事项

- 不要把真实 API key、token、密码写入 Markdown、Word、PPT、日志或 Git。
- `certs/\*.key` 是私钥，不能公开分享。
- 代理日志公开前要扫敏感头，例如 `authorization`、`x-api-key`、`api-key`。
- 不同 Mac 的 `~/Library/Application Support/Claude-3p/claude-code/<version>` 版本号可能不同，按日志为准。
- 如果是公司内网批量部署，优先使用 MDM/配置描述文件分发 CA，而不是让每台机器手动点信任。